

实习二：数据库约束设计

吕杭州 2200013126

戴思颖 2200094811

戴傅聪 2100013061

2025 年 4 月 21 日

本次实习的目标是请同学们体验如何在数据库中利用各种手段完成数据库约束设计。完善的约束设计功能使得数据一致性维护以及业务规则甚至异常处理都可以统一在服务器端完成，减轻了应用处理的负担。同学们在实现各个约束的时候，应该同时给出正负测试样例，也即符合约束以及违反约束的增删改操作。

具体的约束设计任务我们分为基础、中级、高级、扩展四部分，如下：

1. 基本约束设计

我们需要为以下两个表设计约束

- Emp(eno, ename, birthday, level, position, salary, dno)
- Dept(dno, dname, budget, manager)

约束要求：

1. eno 和 dno 是递增序列号形式的主键，长度为 4 的整型，格式为 0001、0002 等
2. Emp 中的 dno 为参照 Dept 的外键，Dept 的 manager 为参照 Emp 的外键
3. 测试外键定义的三种形式
4. 限定 dname 为枚举类型（数学学院、计算机学院、智能学院、电子学院、元培学院）
5. 限定 position 为枚举类型（教师、教务、会计、秘书）
6. 限定 level 为 1 到 5，默认值为 3，salary 为 2000 到 200000

```
1 import os
2 from sqlalchemy import create_engine, text, MetaData, Column, Integer, String, Float,
  Date, ForeignKey, CheckConstraint
3 from sqlalchemy.ext.declarative import declarative_base
4 from sqlalchemy.orm import sessionmaker, relationship
5 from datetime import date
```

```
1 # 创建SQLite数据库连接
2 engine = create_engine('sqlite:///employee_dept.db', echo=True)
3 metadata = MetaData()
4 Base = declarative_base()
5
6 # 创建会话
7 Session = sessionmaker(bind=engine)
8 session = Session()
```

C:\Users\Dai\AppData\Local\Temp\ipykernel_14120\1065261567.py:4: MovedIn20Warning: The ``declarative\_base()`` function is now available as sqlalchemy.orm.declarative_base(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: <https://sqlalche.me/e/b8d9>)

```
Base = declarative_base()
```

```
1 # 如果已经存在数据库，删除它
2 if os.path.exists('employee_dept.db'):
3     os.remove('employee_dept.db')
4     print("删除现有数据库文件")
```

删除现有数据库文件

```
1 # 定义Emp表（放在前面，因为Dept表需要引用它）
2 class Emp(Base):
3     __tablename__ = 'emp' # 修正了双下划线
4
5     eno = Column(String(4), primary_key=True)
```

```

6     ename = Column(String(50))
7     birthday = Column(Date)
8     level = Column(Integer, CheckConstraint("level BETWEEN 1 AND 5"), default=3)
9     position = Column(String(10), CheckConstraint("position IN ('教师', '教务', '会计', '秘书')"))
10    salary = Column(Float, CheckConstraint("salary BETWEEN 2000 AND 200000"))
11    dno = Column(String(4), ForeignKey('dept.dno', deferrable=True,
initially='DEFERRED'))
12
13    def __repr__(self):
14        return f"<Emp(eno='{self.eno}', ename='{self.ename}', level={self.level},
position='{self.position}', salary={self.salary}, dno='{self.dno}')>"
15
16    # 注意：我们将在Dept类定义后添加关系
17
18    # 定义Dept表
19    class Dept(Base):
20        __tablename__ = 'dept' # 修正了双下划线
21
22        dno = Column(String(4), primary_key=True)
23        dname = Column(String(20), CheckConstraint("dname IN ('数学学院', '计算机学院', '智
能学院', '电子学院', '元培学院')"))
24        budget = Column(Float)
25        manager = Column(String(4), ForeignKey('emp.eno', deferrable=True,
initially='DEFERRED'))
26
27        def __repr__(self):
28            return f"<Dept(dno='{self.dno}', dname='{self.dname}', budget={self.budget},
manager='{self.manager}')>"
29
30    # 添加关系引用，解决循环引用问题
31    Emp.department = relationship("Dept", foreign_keys=[Emp.dno], backref="employees")
32    Dept.manager_emp = relationship("Emp", foreign_keys=[Dept.manager])

```

```
1 # 创建表
```

python

```
2 Base.metadata.create_all(engine)
```

```

2025-04-19 12:11:39,389 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:39,391 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("emp")
2025-04-19 12:11:39,391 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-04-19 12:11:39,393 INFO sqlalchemy.engine.Engine PRAGMA temp.table_info("emp")
2025-04-19 12:11:39,393 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-04-19 12:11:39,395 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("dept")
2025-04-19 12:11:39,395 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-04-19 12:11:39,396 INFO sqlalchemy.engine.Engine PRAGMA temp.table_info("dept")
2025-04-19 12:11:39,397 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-04-19 12:11:39,399 INFO sqlalchemy.engine.Engine
CREATE TABLE emp (
  eno VARCHAR(4) NOT NULL,
  ename VARCHAR(50),
  birthday DATE,
  level INTEGER CHECK (level BETWEEN 1 AND 5),
  position VARCHAR(10) CHECK (position IN ('教师', '教务', '会计', '秘书')),
  salary FLOAT CHECK (salary BETWEEN 2000 AND 200000),
  dno VARCHAR(4),
  PRIMARY KEY (eno),
  FOREIGN KEY(dno) REFERENCES dept (dno) DEFERRABLE INITIALLY DEFERRED

```

)

2025-04-19 12:11:39,399 INFO sqlalchemy.engine.Engine [no key 0.00049s] ()

2025-04-19 12:11:39,452 INFO sqlalchemy.engine.Engine

```
CREATE TABLE dept (  
    dno VARCHAR(4) NOT NULL,  
    dname VARCHAR(20) CHECK (dname IN ('数学学院', '计算机学院', '智能学院', '电子学院', '元培  
学院')),  
    budget FLOAT,  
    manager VARCHAR(4),  
    PRIMARY KEY (dno),  
    FOREIGN KEY(manager) REFERENCES emp (eno) DEFERRABLE INITIALLY DEFERRED  
)
```

2025-04-19 12:11:39,454 INFO sqlalchemy.engine.Engine [no key 0.00267s] ()

2025-04-19 12:11:39,471 INFO sqlalchemy.engine.Engine COMMIT

```
1 # 定义测试函数  
2 def run_test(test_name, test_func):  
3     """运行测试并打印结果"""  
4     print(f"\n测试: {test_name}")  
5     print("-" * 50)  
6     try:  
7         test_func()  
8         print("✓ 测试通过")  
9     except Exception as e:  
10        print(f"× 测试失败: {e}")  
11    finally:  
12        session.rollback()
```

python

```
1 # 为SQLite创建序列模拟功能  
2 def get_next_id(table_name):  
3     max_id_query = text(f"SELECT MAX(CAST(SUBSTR({table_name[0]}no, 1, 4) AS INTEGER))  
FROM {table_name}")  
4     result = engine.execute(max_id_query).scalar()  
5     next_id = 1 if result is None else result + 1  
6     return f"{next_id:04d}"
```

python

```
1 # 插入初始数据  
2 try:  
3     session.begin()  
4     dept1 = Dept(dno="0001", dname="计算机学院", budget=1000000)  
5     dept2 = Dept(dno="0002", dname="数学学院", budget=800000)  
6     dept3 = Dept(dno="0003", dname="智能学院", budget=1200000)  
7     session.add_all([dept1, dept2, dept3])  
8     session.flush()  
9  
10    emp1 = Emp(eno="0001", ename="张三", birthday=date(1980, 1, 15), level=4,  
position="教师", salary=15000, dno="0001")  
11    emp2 = Emp(eno="0002", ename="李四", birthday=date(1985, 3, 20), level=3,  
position="教务", salary=8000, dno="0002")  
12    emp3 = Emp(eno="0003", ename="王五", birthday=date(1990, 7, 10), level=5,  
position="教师", salary=20000, dno="0003")  
13    session.add_all([emp1, emp2, emp3])
```

python

```

14     session.flush()
15
16     dept1.manager = "0001"
17     dept2.manager = "0002"
18     dept3.manager = "0003"
19
20     session.commit()
21 except Exception as e:
22     session.rollback()
23     print(f"插入数据时出错: {e}")

```

```

2025-04-19 12:11:41,364 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:41,366 INFO sqlalchemy.engine.Engine INSERT INTO dept (dno, dname, budget,
manager) VALUES (?, ?, ?, ?)
2025-04-19 12:11:41,367 INFO sqlalchemy.engine.Engine [generated in 0.00080s] [('0001', '计
算机学院', 1000000.0, None), ('0002', '数学学院', 800000.0, None), ('0003', '智能学院',
1200000.0, None)]
2025-04-19 12:11:41,370 INFO sqlalchemy.engine.Engine INSERT INTO emp (eno, ename, birthday,
level, position, salary, dno) VALUES (?, ?, ?, ?, ?, ?, ?)
2025-04-19 12:11:41,370 INFO sqlalchemy.engine.Engine [generated in 0.00069s] [('0001', '张
三', '1980-01-15', 4, '教师', 15000.0, '0001'), ('0002', '李四', '1985-03-20', 3, '教务',
8000.0, '0002'), ('0003', '王五', '1990-07-10', 5, '教师', 20000.0, '0003')]
2025-04-19 12:11:41,374 INFO sqlalchemy.engine.Engine UPDATE dept SET manager=? WHERE dept.dno
= ?
2025-04-19 12:11:41,375 INFO sqlalchemy.engine.Engine [generated in 0.00123s] [('0001',
'0001'), ('0002', '0002'), ('0003', '0003')]
2025-04-19 12:11:41,376 INFO sqlalchemy.engine.Engine COMMIT

```

```

1  # 测试默认值 python
2  try:
3      session.begin()
4      emp_default = Emp(eno="0004", ename="吴十", birthday=date(1991, 8, 25), position="秘
书", salary=7000, dno="0003")
5      session.add(emp_default)
6      session.commit()
7      emp_result = session.query(Emp).filter_by(ename="吴十").first()
8      print(f"默认值测试结果: {emp_result.level}")
9  except Exception as e:
10     session.rollback()
11     print(f"测试默认值时出错: {e}")

```

```

2025-04-19 12:11:42,175 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:42,177 INFO sqlalchemy.engine.Engine INSERT INTO emp (eno, ename, birthday,
level, position, salary, dno) VALUES (?, ?, ?, ?, ?, ?, ?)
2025-04-19 12:11:42,177 INFO sqlalchemy.engine.Engine [generated in 0.00059s] ('0004', '吴十',
'1991-08-25', 3, '秘书', 7000.0, '0003')
2025-04-19 12:11:42,192 INFO sqlalchemy.engine.Engine COMMIT
2025-04-19 12:11:42,207 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:42,209 INFO sqlalchemy.engine.Engine SELECT emp.eno AS emp_eno, emp.ename AS
emp_ename, emp.birthday AS emp_birthday, emp.level AS emp_level, emp.position AS emp_position,
emp.salary AS emp_salary, emp.dno AS emp_dno
FROM emp
WHERE emp.ename = ?
LIMIT ? OFFSET ?
2025-04-19 12:11:42,209 INFO sqlalchemy.engine.Engine [generated in 0.00060s] ('吴十', 1, 0)
默认值测试结果: 3

```

```

1  # 正向测试用例 python
2  def test_valid_emp_insert():

```

```
3      """测试正常员工数据插入"""
4      emp = Emp(
5          eno="0006",
6          ename="吴六",
7          birthday=date(1990, 5, 15),
8          level=2,
9          position="教师",
10         salary=12000,
11         dno="0001"
12     )
13     session.add(emp)
14     session.commit()
15
16     # 验证插入成功
17     result = session.query(Emp).filter_by(eno="0003").first()
18     assert result is not None, "员工数据未成功插入"
19     print(f"成功插入员工: {result}")
20
21     run_test("正常员工数据插入", test_valid_emp_insert)
```

测试：正常员工数据插入

```
-----
2025-04-19 12:11:43,735 INFO sqlalchemy.engine.Engine INSERT INTO emp (eno, ename, birthday,
level, position, salary, dno) VALUES (?, ?, ?, ?, ?, ?, ?)
2025-04-19 12:11:43,736 INFO sqlalchemy.engine.Engine [generated in 0.00111s] ('0006', '吴六',
'1990-05-15', 2, '教师', 12000.0, '0001')
2025-04-19 12:11:43,741 INFO sqlalchemy.engine.Engine COMMIT
2025-04-19 12:11:43,755 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:43,758 INFO sqlalchemy.engine.Engine SELECT emp.eno AS emp_eno, emp.ename AS
emp_ename, emp.birthday AS emp_birthday, emp.level AS emp_level, emp.position AS emp_position,
emp.salary AS emp_salary, emp.dno AS emp_dno
FROM emp
WHERE emp.eno = ?
LIMIT ? OFFSET ?
2025-04-19 12:11:43,758 INFO sqlalchemy.engine.Engine [generated in 0.00128s] ('0003', 1, 0)
成功插入员工: <Emp(eno='0003', ename='王五', level=5, position='教师', salary=20000.0,
dno='0003')>
✓ 测试通过
2025-04-19 12:11:43,762 INFO sqlalchemy.engine.Engine ROLLBACK
```

```
1  def test_valid_dept_insert():
2      """测试正常部门数据插入"""
3      dept = Dept(
4          dno="0005",
5          dname="电子学院",
6          budget=1200000,
7          manager="0004"
8      )
9      session.add(dept)
10     session.commit()
11
12     # 验证插入成功
13     result = session.query(Dept).filter_by(dno="0003").first()
14     assert result is not None, "部门数据未成功插入"
15     print(f"成功插入部门: {result}")
```

python

```
16
17 run_test("正常部门数据插入", test_valid_dept_insert)
```

测试：正常部门数据插入

```
-----
2025-04-19 12:11:44,521 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:44,522 INFO sqlalchemy.engine.Engine INSERT INTO dept (dno, dname, budget,
manager) VALUES (?, ?, ?, ?)
2025-04-19 12:11:44,522 INFO sqlalchemy.engine.Engine [generated in 0.00065s] ('0005', '电子学
院', 1200000.0, '0004')
2025-04-19 12:11:44,525 INFO sqlalchemy.engine.Engine COMMIT
2025-04-19 12:11:44,552 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:44,553 INFO sqlalchemy.engine.Engine SELECT dept.dno AS dept_dno, dept.dname
AS dept_dname, dept.budget AS dept_budget, dept.manager AS dept_manager
FROM dept
WHERE dept.dno = ?
LIMIT ? OFFSET ?
2025-04-19 12:11:44,553 INFO sqlalchemy.engine.Engine [generated in 0.00117s] ('0003', 1, 0)
成功插入部门: <Dept(dno='0003', dname='智能学院', budget=1200000.0, manager='0003')>
✓ 测试通过
2025-04-19 12:11:44,555 INFO sqlalchemy.engine.Engine ROLLBACK
```

```
1 def test_default_level():
2     """测试员工level默认值为3"""
3     emp = Emp(
4         eno="0005",
5         ename="赵六",
6         birthday=date(1992, 8, 25),
7         position="秘书",
8         salary=7000,
9         dno="0001"
10    )
11    session.add(emp)
12    session.commit()
13
14    # 验证默认值
15    result = session.query(Emp).filter_by(eno="0005").first()
16    assert result.level == 3, f"默认值错误: 预期为3, 实际为{result.level}"
17    print(f"默认值测试成功: {result}")
18
19 run_test("员工level默认值测试", test_default_level)
```

测试：员工level默认值测试

```
-----
2025-04-19 12:11:45,163 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:45,165 INFO sqlalchemy.engine.Engine INSERT INTO emp (eno, ename, birthday,
level, position, salary, dno) VALUES (?, ?, ?, ?, ?, ?, ?)
2025-04-19 12:11:45,166 INFO sqlalchemy.engine.Engine [cached since 2.99s ago] ('0005', '赵
六', '1992-08-25', 3, '秘书', 7000.0, '0001')
2025-04-19 12:11:45,172 INFO sqlalchemy.engine.Engine COMMIT
2025-04-19 12:11:45,202 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:45,205 INFO sqlalchemy.engine.Engine SELECT emp.eno AS emp_eno, emp.ename AS
emp_ename, emp.birthday AS emp_birthday, emp.level AS emp_level, emp.position AS emp_position,
emp.salary AS emp_salary, emp.dno AS emp_dno
FROM emp
WHERE emp.eno = ?
LIMIT ? OFFSET ?
```

2025-04-19 12:11:45,206 INFO sqlalchemy.engine.Engine [cached since 1.448s ago] ('0005', 1, 0)
默认值测试成功: <Emp(eno='0005', ename='赵六', level=3, position='秘书', salary=7000.0, dno='0001')>

✓ 测试通过

2025-04-19 12:11:45,207 INFO sqlalchemy.engine.Engine ROLLBACK

```

1  def test_circular_reference():
2      """ 测试循环引用 - 员工引用部门，部门经理引用员工 """
3      # 先创建部门，不设置经理
4      dept = Dept(dno="0004", dname="元培学院", budget=900000)
5      session.add(dept)
6      session.flush()
7
8      # 创建员工，引用该部门
9      emp = Emp(
10         eno="0007",
11         ename="钱七",
12         birthday=date(1988, 9, 12),
13         level=5,
14         position="教师",
15         salary=25000,
16         dno="0004"
17     )
18     session.add(emp)
19     session.flush()
20
21     # 将该员工设为部门经理
22     dept.manager = "0007"
23     session.commit()
24
25     # 验证引用关系
26     dept_result = session.query(Dept).filter_by(dno="0004").first()
27     emp_result = session.query(Emp).filter_by(eno="0007").first()
28
29     assert dept_result.manager == "0007", "部门经理未设置成功"
30     assert emp_result.dno == "0004", "员工部门未设置成功"
31     print(f"循环引用测试成功: 部门{dept_result}, 员工{emp_result}")
32
33 run_test("循环引用测试", test_circular_reference)

```

测试：循环引用测试

```

-----
2025-04-19 12:11:45,812 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:45,812 INFO sqlalchemy.engine.Engine INSERT INTO dept (dno, dname, budget,
manager) VALUES (?, ?, ?, ?)
2025-04-19 12:11:45,814 INFO sqlalchemy.engine.Engine [cached since 1.292s ago] ('0004', '元培
学院', 900000.0, None)
2025-04-19 12:11:45,820 INFO sqlalchemy.engine.Engine INSERT INTO emp (eno, ename, birthday,
level, position, salary, dno) VALUES (?, ?, ?, ?, ?, ?, ?)
2025-04-19 12:11:45,821 INFO sqlalchemy.engine.Engine [cached since 2.086s ago] ('0007', '钱
七', '1988-09-12', 5, '教师', 25000.0, '0004')
2025-04-19 12:11:45,823 INFO sqlalchemy.engine.Engine UPDATE dept SET manager=? WHERE dept.dno
= ?
2025-04-19 12:11:45,824 INFO sqlalchemy.engine.Engine [generated in 0.00094s] ('0007', '0004')
2025-04-19 12:11:45,825 INFO sqlalchemy.engine.Engine COMMIT
2025-04-19 12:11:45,834 INFO sqlalchemy.engine.Engine BEGIN (implicit)

```

```

2025-04-19 12:11:45,836 INFO sqlalchemy.engine.Engine SELECT dept.dno AS dept_dno, dept.dname
AS dept_dname, dept.budget AS dept_budget, dept.manager AS dept_manager
FROM dept
WHERE dept.dno = ?
LIMIT ? OFFSET ?
2025-04-19 12:11:45,836 INFO sqlalchemy.engine.Engine [cached since 1.284s ago] ('0004', 1, 0)
2025-04-19 12:11:45,840 INFO sqlalchemy.engine.Engine SELECT emp.eno AS emp_eno, emp.ename AS
emp_ename, emp.birthday AS emp_birthday, emp.level AS emp_level, emp.position AS emp_position,
emp.salary AS emp_salary, emp.dno AS emp_dno
FROM emp
WHERE emp.eno = ?
LIMIT ? OFFSET ?
2025-04-19 12:11:45,840 INFO sqlalchemy.engine.Engine [cached since 2.083s ago] ('0007', 1, 0)
循环引用测试成功: 部门<Dept(dno='0004', dname='元培学院', budget=900000.0, manager='0007')>,
员工<Emp(eno='0007', ename='钱七', level=5, position='教师', salary=25000.0, dno='0004')>
✓ 测试通过
2025-04-19 12:11:45,840 INFO sqlalchemy.engine.Engine ROLLBACK

```

```

1  # 负向测试用例
2  def test_invalid_level():
3      """测试员工level约束 (1-5之间)"""
4      emp = Emp(
5          eno="0009",
6          ename="测试",
7          birthday=date(1990, 5, 15),
8          level=6, # 超出范围
9          position="教师",
10         salary=12000,
11         dno="0001"
12     )
13     session.add(emp)
14     session.commit() # 应该引发异常
15
16 run_test("非法员工level值(超过上限)", test_invalid_level)

```

测试: 非法员工level值(超过上限)

```

-----
2025-04-19 12:11:46,653 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:46,653 INFO sqlalchemy.engine.Engine INSERT INTO emp (eno, ename, birthday,
level, position, salary, dno) VALUES (?, ?, ?, ?, ?, ?, ?)
2025-04-19 12:11:46,653 INFO sqlalchemy.engine.Engine [cached since 2.918s ago] ('0009', '测
试', '1990-05-15', 6, '教师', 12000.0, '0001')
2025-04-19 12:11:46,655 INFO sqlalchemy.engine.Engine ROLLBACK
× 测试失败: (sqlite3.IntegrityError) CHECK constraint failed: level BETWEEN 1 AND 5
[SQL: INSERT INTO emp (eno, ename, birthday, level, position, salary, dno) VALUES
(?, ?, ?, ?, ?, ?, ?)]
[parameters: ('0009', '测试', '1990-05-15', 6, '教师', 12000.0, '0001')]
(Background on this error at: https://sqlalche.me/e/20/gkpj)

```

```

1  def test_invalid_level_low():
2      """测试员工level下限约束"""
3      emp = Emp(
4          eno="0009",
5          ename="测试",
6          birthday=date(1990, 5, 15),
7          level=0, # 低于下限
8          position="教师",

```



```

9         salary=12000,
10         dno="0001"
11     )
12     session.add(emp)
13     session.commit() # 应该引发异常
14 run_test("非法员工level值(低于下限)", test_invalid_level)

```

测试：非法员工level值(低于下限)

```

-----
2025-04-19 12:11:47,467 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:47,468 INFO sqlalchemy.engine.Engine INSERT INTO emp (eno, ename, birthday,
level, position, salary, dno) VALUES (?, ?, ?, ?, ?, ?, ?)
2025-04-19 12:11:47,468 INFO sqlalchemy.engine.Engine [cached since 3.733s ago] ('0009', '测试',
'1990-05-15', 6, '教师', 12000.0, '0001')
2025-04-19 12:11:47,469 INFO sqlalchemy.engine.Engine ROLLBACK
× 测试失败: (sqlite3.IntegrityError) CHECK constraint failed: level BETWEEN 1 AND 5
[SQL: INSERT INTO emp (eno, ename, birthday, level, position, salary, dno) VALUES
(?, ?, ?, ?, ?, ?, ?)]
[parameters: ('0009', '测试', '1990-05-15', 6, '教师', 12000.0, '0001')]
(Background on this error at: https://sqlalche.me/e/20/gkpj)

```

```

1  def test_invalid_position():
2      """测试职位枚举约束"""
3      emp = Emp(
4          eno="0009",
5          ename="测试",
6          birthday=date(1990, 5, 15),
7          level=3,
8          position="主任", # 不在枚举列表中
9          salary=12000,
10         dno="0001"
11     )
12     session.add(emp)
13     session.commit() # 应该引发异常
14 run_test("非法职位值", test_invalid_position)

```

python

测试：非法职位值

```

-----
2025-04-19 12:11:48,525 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:48,526 INFO sqlalchemy.engine.Engine INSERT INTO emp (eno, ename, birthday,
level, position, salary, dno) VALUES (?, ?, ?, ?, ?, ?, ?)
2025-04-19 12:11:48,527 INFO sqlalchemy.engine.Engine [cached since 4.791s ago] ('0009', '测试',
'1990-05-15', 3, '主任', 12000.0, '0001')
2025-04-19 12:11:48,527 INFO sqlalchemy.engine.Engine ROLLBACK
× 测试失败: (sqlite3.IntegrityError) CHECK constraint failed: position IN ('教师', '教务', '会计', '秘书')
[SQL: INSERT INTO emp (eno, ename, birthday, level, position, salary, dno) VALUES
(?, ?, ?, ?, ?, ?, ?)]
[parameters: ('0009', '测试', '1990-05-15', 3, '主任', 12000.0, '0001')]
(Background on this error at: https://sqlalche.me/e/20/gkpj)

```

```

1  def test_invalid_salary_low():
2      """测试薪资下限约束"""
3      emp = Emp(
4          eno="0009",
5          ename="测试",

```

python

```

6         birthday=date(1990, 5, 15),
7         level=3,
8         position="教师",
9         salary=1000, # 低于最低薪资
10        dno="0001"
11    )
12    session.add(emp)
13    session.commit() # 应该引发异常
14    run_test("非法薪资值(低于下限)", test_invalid_salary_low)

```

测试：非法薪资值(低于下限)

```

-----
2025-04-19 12:11:50,138 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:50,139 INFO sqlalchemy.engine.Engine INSERT INTO emp (eno, ename, birthday,
level, position, salary, dno) VALUES (?, ?, ?, ?, ?, ?, ?)
2025-04-19 12:11:50,139 INFO sqlalchemy.engine.Engine [cached since 6.404s ago] ('0009', '测试',
'1990-05-15', 3, '教师', 1000.0, '0001')
2025-04-19 12:11:50,141 INFO sqlalchemy.engine.Engine ROLLBACK
× 测试失败: (sqlite3.IntegrityError) CHECK constraint failed: salary BETWEEN 2000 AND 200000
[SQL: INSERT INTO emp (eno, ename, birthday, level, position, salary, dno) VALUES
(?, ?, ?, ?, ?, ?, ?)]
[parameters: ('0009', '测试', '1990-05-15', 3, '教师', 1000.0, '0001')]
(Background on this error at: https://sqlalche.me/e/20/gkpj)

```

```

1  def test_invalid_salary_high():
2      """测试薪资上限约束"""
3      emp = Emp(
4          eno="0006",
5          ename="测试",
6          birthday=date(1990, 5, 15),
7          level=3,
8          position="教师",
9          salary=250000, # 高于最高薪资
10         dno="0001"
11     )
12     session.add(emp)
13     session.commit() # 应该引发异常
14     run_test("非法薪资值(超过上限)", test_invalid_salary_low)

```

python

测试：非法薪资值(超过上限)

```

-----
2025-04-19 12:11:51,167 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:51,167 INFO sqlalchemy.engine.Engine INSERT INTO emp (eno, ename, birthday,
level, position, salary, dno) VALUES (?, ?, ?, ?, ?, ?, ?)
2025-04-19 12:11:51,168 INFO sqlalchemy.engine.Engine [cached since 7.433s ago] ('0009', '测试',
'1990-05-15', 3, '教师', 1000.0, '0001')
2025-04-19 12:11:51,168 INFO sqlalchemy.engine.Engine ROLLBACK
× 测试失败: (sqlite3.IntegrityError) CHECK constraint failed: salary BETWEEN 2000 AND 200000
[SQL: INSERT INTO emp (eno, ename, birthday, level, position, salary, dno) VALUES
(?, ?, ?, ?, ?, ?, ?)]
[parameters: ('0009', '测试', '1990-05-15', 3, '教师', 1000.0, '0001')]
(Background on this error at: https://sqlalche.me/e/20/gkpj)

```

```

1  def test_invalid_dname():
2      """测试部门名称枚举约束"""
3      dept = Dept(

```

python

```

4         dno="0009",
5         dname="物理学院", # 不在枚举列表中
6         budget=1000000,
7         manager="0001"
8     )
9     session.add(dept)
10    session.commit() # 应该引发异常
11
12    run_test("非法部门名称", test_invalid_dname)

```

测试：非法部门名称

```

-----
2025-04-19 12:11:52,124 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:52,124 INFO sqlalchemy.engine.Engine INSERT INTO dept (dno, dname, budget,
manager) VALUES (?, ?, ?, ?)
2025-04-19 12:11:52,125 INFO sqlalchemy.engine.Engine [cached since 7.603s ago] ('0009', '物理
学院', 1000000.0, '0001')
2025-04-19 12:11:52,126 INFO sqlalchemy.engine.Engine ROLLBACK
× 测试失败: (sqlite3.IntegrityError) CHECK constraint failed: dname IN ('数学学院', '计算机学
院', '智能学院', '电子学院', '元培学院')
[SQL: INSERT INTO dept (dno, dname, budget, manager) VALUES (?, ?, ?, ?)]
[parameters: ('0009', '物理学院', 1000000.0, '0001')]
(Background on this error at: https://sqlalche.me/e/20/gkpj)

```

```

1  def test_unique_key_emp():
2      """测试员工主键唯一性约束"""
3      # 插入已存在的员工编号
4      emp = Emp(
5          eno="0001", # 已存在的ID
6          ename="重复",
7          birthday=date(1995, 5, 15),
8          level=3,
9          position="教师",
10         salary=12000,
11         dno="0001"
12     )
13     session.add(emp)
14     session.commit() # 应该引发异常
15
16    run_test("重复员工主键", test_unique_key_emp)

```

python

测试：重复员工主键

```

-----
2025-04-19 12:11:53,144 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:53,144 INFO sqlalchemy.engine.Engine SELECT emp.eno AS emp_eno, emp.ename AS
emp_ename, emp.birthday AS emp_birthday, emp.level AS emp_level, emp.position AS emp_position,
emp.salary AS emp_salary, emp.dno AS emp_dno
FROM emp
WHERE emp.eno = ?
2025-04-19 12:11:53,146 INFO sqlalchemy.engine.Engine [generated in 0.00069s] ('0001',)
2025-04-19 12:11:53,148 INFO sqlalchemy.engine.Engine INSERT INTO emp (eno, ename, birthday,
level, position, salary, dno) VALUES (?, ?, ?, ?, ?, ?, ?)
2025-04-19 12:11:53,149 INFO sqlalchemy.engine.Engine [cached since 9.413s ago] ('0001', '重
复', '1995-05-15', 3, '教师', 12000.0, '0001')
2025-04-19 12:11:53,149 INFO sqlalchemy.engine.Engine ROLLBACK
× 测试失败: (sqlite3.IntegrityError) UNIQUE constraint failed: emp.eno

```

```
[SQL: INSERT INTO emp (eno, ename, birthday, level, position, salary, dno) VALUES
(?, ?, ?, ?, ?, ?, ?)]
[parameters: ('0001', '重复', '1995-05-15', 3, '教师', 12000.0, '0001')]
(Background on this error at: https://sqlalche.me/e/20/gkpj)
C:\Users\Dai\AppData\Local\Temp\ipykernel_14120\3584338704.py:14: SAWarning: New instance <Emp
at 0x1c59d6b02e0> with identity key (<class '__main__.Emp'>, ('0001',), None) conflicts with
persistent instance <Emp at 0x1c59d63e920>
    session.commit() # 应该引发异常
```

```
1 def test_unique_key_dept():
2     """测试部门主键唯一性约束"""
3     # 插入已存在的部门编号
4     dept = Dept(
5         dno="0001", # 已存在的ID
6         dname="计算机学院",
7         budget=1000000,
8         manager="0001"
9     )
10    session.add(dept)
11    session.commit() # 应该引发异常
12
13 run_test("重复部门主键", test_unique_key_dept)
```

测试：重复部门主键

```
-----
2025-04-19 12:11:55,080 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:55,081 INFO sqlalchemy.engine.Engine SELECT dept.dno AS dept_dno, dept.dname
AS dept_dname, dept.budget AS dept_budget, dept.manager AS dept_manager
FROM dept
WHERE dept.dno = ?
2025-04-19 12:11:55,082 INFO sqlalchemy.engine.Engine [generated in 0.00078s] ('0001',)
2025-04-19 12:11:55,084 INFO sqlalchemy.engine.Engine INSERT INTO dept (dno, dname, budget,
manager) VALUES (?, ?, ?, ?)
2025-04-19 12:11:55,084 INFO sqlalchemy.engine.Engine [cached since 10.56s ago] ('0001', '计算
机学院', 1000000.0, '0001')
2025-04-19 12:11:55,086 INFO sqlalchemy.engine.Engine ROLLBACK
x 测试失败: (sqlite3.IntegrityError) UNIQUE constraint failed: dept.dno
[SQL: INSERT INTO dept (dno, dname, budget, manager) VALUES (?, ?, ?, ?)]
[parameters: ('0001', '计算机学院', 1000000.0, '0001')]
(Background on this error at: https://sqlalche.me/e/20/gkpj)
C:\Users\Dai\AppData\Local\Temp\ipykernel_14120\1215435416.py:11: SAWarning: New instance
<Dept at 0x1c59d6b1990> with identity key (<class '__main__.Dept'>, ('0001',), None) conflicts
with persistent instance <Dept at 0x1c59d6b3d30>
    session.commit() # 应该引发异常
```

```
1 # 查询部门数据
2 for dept in session.query(Dept).all():
3     print(dept)
```

```
2025-04-19 12:11:55,973 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-19 12:11:55,975 INFO sqlalchemy.engine.Engine SELECT dept.dno AS dept_dno, dept.dname
AS dept_dname, dept.budget AS dept_budget, dept.manager AS dept_manager
FROM dept
2025-04-19 12:11:55,976 INFO sqlalchemy.engine.Engine [generated in 0.00065s] ()
<Dept(dno='0001', dname='计算机学院', budget=1000000.0, manager='0001')>
<Dept(dno='0002', dname='数学学院', budget=800000.0, manager='0002')>
<Dept(dno='0003', dname='智能学院', budget=1200000.0, manager='0003')>
<Dept(dno='0005', dname='电子学院', budget=1200000.0, manager='0004')>
<Dept(dno='0004', dname='元培学院', budget=900000.0, manager='0007')>
```

```

1 # 查询员工数据
2 for emp in session.query(Emp).all():
3     print(emp)
4
5 # 清理资源
6 session.close()
7 print("会话已关闭")

```

2025-04-19 12:11:56,820 INFO sqlalchemy.engine.Engine SELECT emp.eno AS emp_eno, emp.ename AS emp_ename, emp.birthday AS emp_birthday, emp.level AS emp_level, emp.position AS emp_position, emp.salary AS emp_salary, emp.dno AS emp_dno
FROM emp

2025-04-19 12:11:56,821 INFO sqlalchemy.engine.Engine [generated in 0.00086s] ()
<Emp(eno='0001', ename='张三', level=4, position='教师', salary=15000.0, dno='0001')>
<Emp(eno='0002', ename='李四', level=3, position='教务', salary=8000.0, dno='0002')>
<Emp(eno='0003', ename='王五', level=5, position='教师', salary=20000.0, dno='0003')>
<Emp(eno='0004', ename='吴十', level=3, position='秘书', salary=7000.0, dno='0003')>
<Emp(eno='0006', ename='吴六', level=2, position='教师', salary=12000.0, dno='0001')>
<Emp(eno='0005', ename='赵六', level=3, position='秘书', salary=7000.0, dno='0001')>
<Emp(eno='0007', ename='钱七', level=5, position='教师', salary=25000.0, dno='0004')>

2025-04-19 12:11:56,822 INFO sqlalchemy.engine.Engine ROLLBACK

会话已关闭

2. 中级约束设计

1. 测试延迟约束，往 Emp 和 Dept 中插入互相参照的两行。PG 是支持延迟约束的，MySQL 不支持延迟约束，可以通过设置约束是否有效来完成
2. 将 salary 划分为 5 个区间，每个区间对应一个 level 值，保证每个员工的工资值和他的 level 值是正确对应的，这属于行级约束
3. 编写函数，输入员工的员工号，输出一个包含员工各方面信息的编码字符串，也即我们第二章中提到的智能码。比如 00010002199903020002，对应编码信息如下：

0001	0002	1999	03	02	0002
员工号	部门号	出生年份	级别编码	职位编码	部门领导号

同学们在实现时，规范的做法是构造一张编码对照表，而不是把编码对应信息直接放在代码里面

```

1 import sqlite3
2 from datetime import date
3
4 def init_database():
5     conn = sqlite3.connect('university.db')
6     cursor = conn.cursor()
7
8     # Enable foreign key support
9     cursor.execute("PRAGMA foreign_keys = ON")
10
11     # Create Emp table
12     cursor.execute('''
13 CREATE TABLE IF NOT EXISTS emp (
14     eno TEXT(4) PRIMARY KEY,
15     ename TEXT(50),
16     birthday DATE,
17     level INTEGER DEFAULT 3 CHECK(level BETWEEN 1 AND 5),
18     position TEXT(10) CHECK(position IN ('教师', '教务', '会计', '秘书')),
19     salary REAL CHECK(salary BETWEEN 2000 AND 200000),

```

```
20     dno TEXT(4),
21     FOREIGN KEY (dno) REFERENCES dept(dno) DEFERRABLE INITIALLY DEFERRED
22 )
23 '''
24
25 # Create Dept table
26 cursor.execute('''
27 CREATE TABLE IF NOT EXISTS dept (
28     dno TEXT(4) PRIMARY KEY,
29     dname TEXT(20) CHECK(dname IN ('数学学院', '计算机学院', '智能学院', '电子学院',
30     '元培学院')),
31     budget REAL,
32     manager TEXT(4),
33     FOREIGN KEY (manager) REFERENCES emp(eno) DEFERRABLE INITIALLY DEFERRED
34 )
35 '''
36
37 # Create code mapping table
38 cursor.execute('''
39 CREATE TABLE IF NOT EXISTS code_mapping (
40     category TEXT NOT NULL,
41     code TEXT NOT NULL,
42     value TEXT NOT NULL,
43     PRIMARY KEY (category, code)
44 )
45 '''
46
47 # Insert code mappings
48 mappings = [
49     # Department codes
50     ('dept', '数学学院', '01'),
51     ('dept', '计算机学院', '02'),
52     ('dept', '智能学院', '03'),
53     ('dept', '电子学院', '04'),
54     ('dept', '元培学院', '05'),
55
56     # Position codes
57     ('position', '教师', '01'),
58     ('position', '教务', '02'),
59     ('position', '会计', '03'),
60     ('position', '秘书', '04'),
61
62     # Level codes (same as value)
63     ('level', '1', '1'),
64     ('level', '2', '2'),
65     ('level', '3', '3'),
66     ('level', '4', '4'),
67     ('level', '5', '5'),
68 ]
69 cursor.executemany(
```

```
70     "INSERT OR IGNORE INTO code_mapping VALUES (?, ?, ?)",
71     mappings
72 )
73
74 conn.commit()
75 return conn

1  def insert_with_circular_reference(conn): python
2      cursor = conn.cursor()
3
4      try:
5          # Start a transaction
6          cursor.execute("BEGIN TRANSACTION")
7
8          # Insert department first (manager will be set later)
9          cursor.execute(
10             "INSERT INTO dept (dno, dname, budget, manager) VALUES (?, ?, ?, ?)",
11             ('D001', '计算机学院', 5000000, None)
12         )
13
14         # Insert employee referencing the department
15         cursor.execute(
16             "INSERT INTO emp (eno, ename, birthday, level, position, salary, dno)
17             VALUES (?, ?, ?, ?, ?, ?, ?)",
18             ('E001', '张教授', date(1975, 5, 15), 4, '教师', 80000, 'D001')
19         )
20
21         # Now update the department to reference the employee as manager
22         cursor.execute(
23             "UPDATE dept SET manager = ? WHERE dno = ?",
24             ('E001', 'D001')
25         )
26
27         conn.commit()
28         print("Successfully inserted circular references")
29
30     except sqlite3.Error as e:
31         conn.rollback()
32         print("Failed to insert circular references:", e)
33
34     def test_constraints(conn):
35         cursor = conn.cursor()
36
37         print("\nTesting valid inserts:")
38         try:
39             # Valid employee
40             cursor.execute(
41                 "INSERT INTO emp (eno, ename, level, position, salary) VALUES
42                 (?, ?, ?, ?, ?)",
43                 ('E002', '李老师', 3, '教师', 50000)
44             )
45             print("- Valid employee inserted")
```

```
44
45     # Valid department
46     cursor.execute(
47         "INSERT INTO dept (dno, dname, budget) VALUES (?, ?, ?)",
48         ('D002', '数学学院', 3000000)
49     )
50     print("- Valid department inserted")
51
52     except sqlite3.Error as e:
53         print("Valid insert failed:", e)
54
55     print("\nTesting invalid inserts:")
56
57     # Invalid level
58     try:
59         cursor.execute(
60             "INSERT INTO emp (eno, ename, level, position, salary) VALUES
61             (?, ?, ?, ?, ?)",
62             ('E003', 'Invalid', 6, '教师', 50000)
63         )
64     except sqlite3.IntegrityError as e:
65         print("- Caught invalid level (6):", e)
66
67     # Invalid position
68     try:
69         cursor.execute(
70             "INSERT INTO emp (eno, ename, level, position, salary) VALUES
71             (?, ?, ?, ?, ?)",
72             ('E004', 'Invalid', 3, '校长', 50000)
73         )
74     except sqlite3.IntegrityError as e:
75         print("- Caught invalid position (校长):", e)
76
77     # Invalid salary
78     try:
79         cursor.execute(
80             "INSERT INTO emp (eno, ename, level, position, salary) VALUES
81             (?, ?, ?, ?, ?)",
82             ('E005', 'Invalid', 3, '教师', 1000)
83         )
84     except sqlite3.IntegrityError as e:
85         print("- Caught invalid salary (1000):", e)
86
87     # Invalid department name
88     try:
89         cursor.execute(
90             "INSERT INTO dept (dno, dname, budget) VALUES (?, ?, ?)",
91             ('D003', '物理学院', 2000000)
92         )
93     except sqlite3.IntegrityError as e:
94         print("- Caught invalid department name (物理学院):", e)
```



```
93     conn.rollback() # Rollback all test data
94
95     def generate_smart_code(conn, eno):
96         """Generate a smart code for an employee based on their information"""
97         cursor = conn.cursor()
98
99         # Get employee information
100        cursor.execute('''
101        SELECT e.eno, e.ename, e.birthday, e.level, e.position, e.salary, e.dno, d.dname
102        FROM emp e LEFT JOIN dept d ON e.dno = d.dno
103        WHERE e.eno = ?
104        ''', (eno,))
105
106        emp = cursor.fetchone()
107        if not emp:
108            return None
109
110        # Get code mappings
111        def get_mapping(category, value):
112            cursor.execute(
113                "SELECT code FROM code_mapping WHERE category = ? AND value = ?",
114                (category, str(value)))
115            result = cursor.fetchone()
116            return result[0] if result else '00'
117
118        # Build smart code parts
119        parts = []
120
121        # 1-4: Employee ID (padded to 4 digits)
122        parts.append(emp[0].zfill(4))
123
124        # 5-6: Department code
125        dept_code = get_mapping('dept', emp[7]) if emp[7] else '00'
126        parts.append(dept_code)
127
128        # 7-10: Birth year
129        birth_year = emp[2][:4] if emp[2] else '0000'
130        parts.append(birth_year)
131
132        # 11-12: Position code
133        pos_code = get_mapping('position', emp[4]) if emp[4] else '00'
134        parts.append(pos_code)
135
136        # 13: Level code
137        level_code = get_mapping('level', emp[3]) if emp[3] else '0'
138        parts.append(level_code)
139
140        # 14-17: Salary grade (salary / 10000)
141        salary_grade = str(int(emp[5] // 10000)).zfill(4) if emp[5] else '0000'
142        parts.append(salary_grade)
143
```

```
144     # Combine all parts
145     return ''.join(parts)
146
147 def test_smart_code(conn):
148     """Test the smart code generation with sample data"""
149     cursor = conn.cursor()
150
151     # Insert test data
152     test_data = [
153         # eno, ename, birthday, level, position, salary, dno
154         ('T001', '王教授', '1980-08-20', 5, '教师', 150000, 'D001'),
155         ('A001', '李会计', '1990-03-15', 3, '会计', 50000, 'D002'),
156         ('S001', '张秘书', '1995-11-05', 2, '秘书', 30000, None),
157     ]
158
159     dept_data = [
160         # dno, dname, budget, manager
161         ('D001', '计算机学院', 5000000, 'T001'),
162         ('D002', '数学学院', 3000000, None),
163     ]
164
165     cursor.executemany(
166         "INSERT OR IGNORE INTO emp VALUES (?, ?, ?, ?, ?, ?, ?)",
167         test_data
168     )
169
170     cursor.executemany(
171         "INSERT OR IGNORE INTO dept VALUES (?, ?, ?, ?)",
172         dept_data
173     )
174
175     conn.commit()
176
177     # Generate and display smart codes
178     print("\nGenerated Smart Codes:")
179     for eno in ['T001', 'A001', 'S001']:
180         code = generate_smart_code(conn, eno)
181         print(f"{eno}: {code}")
182
183     # Decode the smart code
184     if code:
185         print(f"  Decoded: ID={code[:4]}, Dept={code[4:6]}, Birth={code[6:10]}, "
186               f"Position={code[10:12]}, Level={code[12]}, SalaryGrade={code[13:]}")
187
188
189 def test():
190     conn = init_database()
191
192     # Test circular reference insertion
193     print("\n=== Testing Circular Reference ===")
194     insert_with_circular_reference(conn)
```

```

195
196     # Test constraints
197     print("\n=== Testing Constraints ===")
198     test_constraints(conn)
199
200     # Verify the circular reference was successful
201     cursor = conn.cursor()
202     cursor.execute("SELECT * FROM emp WHERE eno = 'E001'")
203     emp = cursor.fetchone()
204     print("\nEmployee E001:", emp)
205
206     cursor.execute("SELECT * FROM dept WHERE dno = 'D001'")
207     dept = cursor.fetchone()
208     print("Department D001:", dept)
209
210     test_smart_code(conn)
211
212     conn.close()

```

```
1 test()
```

python

=== Testing Circular Reference ===

Failed to insert circular references: UNIQUE constraint failed: dept.dno

=== Testing Constraints ===

Testing valid inserts:

- Valid employee inserted
- Valid department inserted

Testing invalid inserts:

- Caught invalid level (6): CHECK constraint failed: level BETWEEN 1 AND 5
- Caught invalid position (校长): CHECK constraint failed: position IN ('教师', '教务', '会计', '秘书')
- Caught invalid salary (1000): CHECK constraint failed: salary BETWEEN 2000 AND 200000
- Caught invalid department name (物理学院): CHECK constraint failed: dname IN ('数学学院', '计算机学院', '智能学院', '电子学院', '元培学院')

Employee E001: ('E001', '张教授', '1975-05-15', 4, '教师', 80000.0, 'D001')

Department D001: ('D001', '计算机学院', 500000.0, 'E001')

Generated Smart Codes:

T001: T0010019800050015

Decoded: ID=T001, Dept=00, Birth=1980, Position=00, Level=5, SalaryGrade=0015

A001: A0010019900030005

Decoded: ID=A001, Dept=00, Birth=1990, Position=00, Level=3, SalaryGrade=0005

S001: S0010019950020003

Decoded: ID=S001, Dept=00, Birth=1995, Position=00, Level=2, SalaryGrade=0003

3. 高级约束设计

1. 使用函数约束或者触发器来保证管理者的工资必须高于他所管理的任何一个员工。
2. 使用触发器保证任何一个员工工资的变化额度，都应该体现在他所在部门的预算上面，本质上这相当于实现了一个物化视图的一致性维护机制（触发器应该考虑到员工改变工作部门的情况，从一致性维护效率的角度，完全重算当然最简单，但希望还是实现基于更新行的增量更新）。

```
1 import sqlite3
```

python

```
1 # 连接到数据库
2 conn = sqlite3.connect('employee_dept.db')
3 cursor = conn.cursor()
```

python

3.1. 创建经理工资检查触发器

创建经理工资检查触发器：更新经理时检查，插入/更新员工时检查。

- check_manager_salary_update.
- check_employee_salary_insert.
- check_employee_salary_update.

所以我们这样来创建触发器，是因为 SQLite3 不支持一些比较高级的 Trigger 特性。

注意：不可能插入经理，只可能修改经理。

```
1 cursor.execute('''
2 DROP TRIGGER IF EXISTS check_manager_salary_update
3 ''')
4 conn.commit()
5 print("已经删除Trigger: check_manager_salary_update")
```

python

已经删除Trigger: check_manager_salary_update

```
1 cursor.execute('''
2 -- 更新时检查经理工资
3 CREATE TRIGGER check_manager_salary_update
4 BEFORE UPDATE OF salary, dno, eno ON Emp
5 FOR EACH ROW
6 WHEN NEW.eno = (SELECT manager FROM Dept WHERE dno = NEW.dno)
7 AND (SELECT IFNULL(MAX(salary), 0)
8 FROM Emp
9 WHERE dno = NEW.dno
10 AND eno ≠ NEW.eno) ≥ NEW.salary
11 BEGIN
12 SELECT RAISE(ABORT, '经理的工资必须高于所有员工');
13 END;
14 ''')
15 conn.commit()
16 print("成功创建更新经理时经理工资检查触发器")
```

python

成功创建更新经理时经理工资检查触发器

```
1 cursor.execute('''
2 DROP TRIGGER IF EXISTS check_employee_salary_insert
3 ''')
4 conn.commit()
5 print("已经删除Trigger: check_employee_salary_insert")
```

python

已经删除Trigger: check_employee_salary_insert

```
1 cursor.execute('''
2 -- 插入时检查员工工资
3 CREATE TRIGGER check_employee_salary_insert
4 BEFORE INSERT ON Emp
5 FOR EACH ROW
6 WHEN NEW.eno ≠ (SELECT manager FROM Dept WHERE dno = NEW.dno)
7 AND NEW.salary ≥ (
```

python

```
8      SELECT salary
9      FROM Emp
10     WHERE eno = (SELECT manager FROM Dept WHERE dno = NEW.dno)
11   )
12 BEGIN
13   SELECT RAISE(ABORT, '员工工资不能高于经理');
14 END;
15 '''
16 conn.commit()
17 print("成功创建插入员工时经理工资检查触发器")
```

成功创建插入员工时经理工资检查触发器

```
1 cursor.execute('''
2 DROP TRIGGER IF EXISTS check_employee_salary_update
3 ''')
4 conn.commit()
5 print("已经删除Trigger: check_employee_salary_update")
```

python

已经删除Trigger: check_employee_salary_update

```
1 cursor.execute('''
2 -- 更新时检查员工工资
3 CREATE TRIGGER check_employee_salary_update
4 BEFORE UPDATE OF salary, dno, eno ON Emp
5 FOR EACH ROW
6 WHEN NEW.eno ≠ (SELECT manager FROM Dept WHERE dno = NEW.dno)
7   AND NEW.salary ≥ (
8     SELECT salary
9     FROM Emp
10    WHERE eno = (SELECT manager FROM Dept WHERE dno = NEW.dno)
11  )
12 BEGIN
13   SELECT RAISE(ABORT, '员工工资不能高于经理');
14 END;
15 '''
16 conn.commit()
17 print("成功创建更新员工时经理工资检查触发器")
```

python

成功创建更新员工时经理工资检查触发器

3.2. 创建预算维护触发器

同样的，分别对于 INSERT、DELETE、UPDATE 创建触发器。但是考虑到 UPDATE 的时候部门变动的情况，我们需要创建两个触发器。

- update_budget_on_insert.
- update_budget_on_delete.
- update_budget_on_dept_change.
- update_budget_on_salary_change.

```
1 cursor.execute('''
2 DROP TRIGGER IF EXISTS update_budget_on_insert
3 ''')
4 conn.commit()
5 print("已经删除Trigger: update_budget_on_insert")
```

python

已经删除Trigger: update_budget_on_insert

```
1 # 创建预算维护触发器 - INSERT python
2 cursor.execute('''
3 CREATE TRIGGER update_budget_on_insert
4 AFTER INSERT ON Emp
5 FOR EACH ROW
6 BEGIN
7     UPDATE Dept SET budget = budget + NEW.salary WHERE dno = NEW.dno;
8 END;
9 ''')
10 conn.commit()
11 print("成功创建插入员工时预算维护触发器")
```

成功创建插入员工时预算维护触发器

```
1 cursor.execute(''' python
2 DROP TRIGGER IF EXISTS update_budget_on_delete
3 ''')
4 conn.commit()
5 print("已经删除Trigger: update_budget_on_delete")
```

已经删除Trigger: update_budget_on_delete

```
1 # 创建预算维护触发器 - DELETE python
2 cursor.execute('''
3 CREATE TRIGGER update_budget_on_delete
4 AFTER DELETE ON Emp
5 FOR EACH ROW
6 BEGIN
7     UPDATE Dept SET budget = budget - OLD.salary WHERE dno = OLD.dno;
8 END;
9 ''')
10 conn.commit()
11 print("成功创建删除员工时预算维护触发器")
```

成功创建删除员工时预算维护触发器

```
1 cursor.execute(''' python
2 DROP TRIGGER IF EXISTS update_budget_on_dept_change
3 ''')
4 conn.commit()
5 print("已经删除Trigger: update_budget_on_dept_change")
```

已经删除Trigger: update_budget_on_dept_change

```
1 # 创建预算维护触发器 - UPDATE when dept changes. python
2 cursor.execute('''
3 -- 部门变更时调整旧/新部门预算
4 CREATE TRIGGER update_budget_on_dept_change
5 AFTER UPDATE OF dno, salary ON Emp
6 FOR EACH ROW
7 WHEN NEW.dno <> OLD.dno
8 BEGIN
9     UPDATE Dept SET budget = budget - OLD.salary WHERE dno = OLD.dno;
10    UPDATE Dept SET budget = budget + NEW.salary WHERE dno = NEW.dno;
11 END;
12 ''')
```

```
13 conn.commit()
14 print("成功创建更新员工为不同部门时预算维护触发器")
```

成功创建更新员工为不同部门时预算维护触发器

```
1 cursor.execute('''
2 DROP TRIGGER IF EXISTS update_budget_on_salary_change
3 ''')
4 conn.commit()
5 print("已经删除Trigger: update_budget_on_salary_change")
```

已经删除Trigger: update_budget_on_salary_change

```
1 # 创建预算维护触发器 - UPDATE when dept NOT changes.
2 cursor.execute('''
3 -- 同部门工资变动时按差值调整
4 CREATE TRIGGER update_budget_on_salary_change
5 AFTER UPDATE OF salary ON Emp
6 FOR EACH ROW
7 WHEN NEW.dno = OLD.dno
8 BEGIN
9     UPDATE Dept
10        SET budget = budget + (NEW.salary - OLD.salary)
11        WHERE dno = NEW.dno;
12 END;
13 ''')
14 conn.commit()
15 print("成功创建更新员工为相同部门时预算维护触发器")
```

成功创建更新员工为相同部门时预算维护触发器

下面我们对于触发器进行测试。

```
1 print("1. 更新经理薪资测试 (应失败): ")
2 try:
3     cursor.execute("UPDATE Emp SET salary = 1000 WHERE eno = '0003';")
4     conn.commit()
5 except sqlite3.DatabaseError as e:
6     print("Error:", e)
```

1. 更新经理薪资测试 (应失败):
Error: 经理的工资必须高于所有员工

```
1 print("1. 更新经理薪资测试 (应成功): ")
2 try:
3     # 提薪至25000
4     cursor.execute("UPDATE Emp SET salary = 25000.0 WHERE eno = '0003'")
5     conn.commit()
6     print("更新经理薪资成功")
7 except sqlite3.DatabaseError as e:
8     print("Error:", e)
```

1. 更新经理薪资测试 (应成功):
更新经理薪资成功

```
1 print("2. 插入员工薪资测试 (应失败): ")
2 try:
```

```
3 cursor.execute("INSERT INTO Emp VALUES ('0009','新人','2000-02-02',2,'秘书',16000.0,'0001');")
4 conn.commit()
5 except sqlite3.DatabaseError as e:
6     print("Error:", e)
```

2. 插入员工薪资测试（应失败）:

Error: 员工工资不能高于经理

```
1 print("2. 插入员工薪资测试（应成功）:")
2 try:
3     # 插入薪资14000, 低于经理
4     cursor.execute("INSERT INTO Emp VALUES ('0009','新教职','2001-12-31',3,'秘书',14000.0,'0001')")
5     conn.commit()
6     print("成功插入员工")
7 except sqlite3.DatabaseError as e:
8     print("Error:", e)
```

2. 插入员工薪资测试（应成功）:

成功插入员工

```
1 print("3. 更新员工薪资测试（应失败）:")
2 try:
3     cursor.execute("UPDATE Emp SET salary = 30000 WHERE eno = '0004';")
4     conn.commit()
5 except sqlite3.DatabaseError as e:
6     print("Error:", e)
```

3. 更新员工薪资测试（应失败）:

Error: 员工工资不能高于经理

```
1 print("3. 更新员工薪资测试（应成功）:")
2 try:
3     # 调薪到6500, 低于经理
4     cursor.execute("UPDATE Emp SET salary = 6500.0 WHERE eno = '0004'")
5     conn.commit()
6     print("更新员工薪资成功")
7 except sqlite3.DatabaseError as e:
8     print("Error:", e)
```

3. 更新员工薪资测试（应成功）:

更新员工薪资成功

```
1 def show_budget(dno):
2     # 使用 ? 占位符, 并通过第二个参数传入参数元组
3     budgets = cursor.execute(
4         "SELECT * FROM Dept WHERE dno = ?",
5         (dno,)
6     ).fetchall()
7     print(f"Dept budgets for {dno}:", budgets)
```

```
1 print("4. 预算 插入触发测试（部门0002预算+6000）:")
2 show_budget('0002')
3 try:
4     cursor.execute("INSERT INTO Emp VALUES ('0008','测试','1999-10-01',2,'教师',6000.0,'0002')")
```



```

5     conn.commit()
6     show_budget('0002')
7 except sqlite3.DatabaserError as e:
8     print("Error:", e)

```

4. 预算 插入触发测试 (部门0002预算+6000):

Dept budgets for 0002: [('0002', '数学学院', 800000.0, '0002')]

Dept budgets for 0002: [('0002', '数学学院', 806000.0, '0002')]

```

1 print("5. 预算 删除触发测试 (部门0002预算-6000): ")
2 show_budget('0002')
3 try:
4     cursor.execute("DELETE FROM Emp WHERE eno = '0008'")
5     conn.commit()
6     show_budget('0002')
7 except sqlite3.DatabaseError as e:
8     print("Error:", e)

```

python

5. 预算 删除触发测试 (部门0002预算-6000):

Dept budgets for 0002: [('0002', '数学学院', 806000.0, '0002')]

Dept budgets for 0002: [('0002', '数学学院', 800000.0, '0002')]

```

1 print("6. 预算 部门变更触发测试 (0004从部门0003更换至部门0002) ")
2 show_budget('0002')
3 show_budget('0003')
4 try:
5     cursor.execute("UPDATE Emp SET dno = '0002' WHERE eno = '0004'")
6     conn.commit()
7     show_budget('0002')
8     show_budget('0003')
9 except sqlite3.DatabaseError as e:
10    print("Error:", e)

```

python

6. 预算 部门变更触发测试 (0004从部门0003更换至部门0002)

Dept budgets for 0002: [('0002', '数学学院', 800000.0, '0002')]

Dept budgets for 0003: [('0003', '智能学院', 1204500.0, '0003')]

Dept budgets for 0002: [('0002', '数学学院', 806500.0, '0002')]

Dept budgets for 0003: [('0003', '智能学院', 1198000.0, '0003')]

```

1 print("7. 预算 薪资修改出发测试 (0004修改薪资为3500) ")
2 show_budget('0002')
3 try:
4     cursor.execute("UPDATE Emp SET salary = 3500 WHERE eno = '0004'")
5     conn.commit()
6     show_budget('0002')
7 except sqlite3.DatabaseError as e:
8     print("Error", e)

```

python

7. 预算 薪资修改出发测试 (0004修改薪资为3500)

Dept budgets for 0002: [('0002', '数学学院', 806500.0, '0002')]

Dept budgets for 0002: [('0002', '数学学院', 803500.0, '0002')]

4. 终极约束设计

my_stock(stock_id, volume, avg_price, profit): 表示所持有的股票编号、数量、持仓平均价格、利润。

trans(trans_id, stock_id, date, price, amount, sell_or_buy): 表示一次交易的编号、股票编号、交易日期、成交价格、成交数量、买入还是卖出。

使用触发器完成下面的工作：

1. 往 `trans` 里面插入一条记录时，根据其是买入还是卖出，调整 `my_stock` 中的 `volume` 以及 `avg_price`。
如果是初次插入的股票交易，就在 `my_stock` 中为该股票新建一条记录，`profit` 置为 0。注意，如果一笔卖出交易的 `amount` 大于 `my_stock` 中该股票的 `volume`，说明是无效的下单交易，应该加以拒绝，直接抛弃。平均价格的计算：

$$\text{avg_price} = \frac{\text{volume} \times \text{avg_price} + \text{price} \times \text{amount}}{\text{volume} + \text{amount}}$$

2. `profit` 的计算方式如下：每当有卖出交易发生时，将其与尽可能远的买入交易进行匹配，比如如果 `trans` 中现有的记录为 `{(t01,s01,d01,10,1000,buy), (t02,s01,d02,12,500,buy)}`，如果现在插入 `{(t03,s01,d03,11,700,sold)}`，本次交易产生的 `profit=(11-10)*700`，如果再插入 `{(t04,s01,d04,9,700,sold)}`，本次交易产生的 `profit=(9-10)*300 + (9-12)*400 = -1500`。将每次卖出交易的 `profit` 都累加到 `my_stock` 的 `profit` 上。

```

1  # 建表
2  import sqlite3
3
4  sqlite3.enable_callback_tracebacks(True)
5
6  conn = sqlite3.connect("stocks")
7  cursor = conn.cursor()
8
9  cursor.execute('''
10 CREATE TABLE IF NOT EXISTS my_stock (
11     stock_id TEXT PRIMARY KEY,
12     volume INTEGER NOT NULL,
13     avg_price REAL NOT NULL,
14     profit REAL NOT NULL DEFAULT 0
15 );
16 ''')
17
18 cursor.execute('''
19 CREATE TABLE IF NOT EXISTS trans (
20     trans_id TEXT PRIMARY KEY,
21     stock_id TEXT NOT NULL REFERENCES my_stock(stock_id),
22     date DATE NOT NULL,
23     price REAL NOT NULL,
24     amount INTEGER NOT NULL,
25     sell_or_buy TEXT NOT NULL CHECK(sell_or_buy IN ('buy','sold'))
26 );
27 ''')
28
29 conn.commit()
30 print("建表完成")

```

建表完成

```

1  # 打印表结构
2  def print_table_schema(table_name, db_path="stocks"):
3      """
4      打印指定 SQLite 数据库中某个表的结构信息。
5
6      参数：

```

```
7         table_name (str): 要查看结构的表名。
8         """
9         try:
10             # 连接到 SQLite 数据库
11             conn = sqlite3.connect(db_path)
12             cursor = conn.cursor()
13
14             # 执行 PRAGMA table_info 命令获取表的结构信息
15             cursor.execute(f"PRAGMA table_info('{table_name}')"")
16             columns = cursor.fetchall()
17
18             if not columns:
19                 print(f"表 '{table_name}' 不存在或没有列信息。")
20                 return
21
22             # 打印表结构信息
23             print(f"表 '{table_name}' 的结构信息: ")
24             print("{:<5} {:<20} {:<15} {:<10} {:<15} {:<5}".format(
25                 "cid", "name", "type", "notnull", "dflt_value", "pk"
26             ))
27             print("-" * 80)
28             for col in columns:
29                 cid, name, col_type, notnull, dflt_value, pk = col
30                 print("{:<5} {:<20} {:<15} {:<10} {:<15} {:<5}".format(
31                     cid, name, col_type, notnull, str(dflt_value), pk
32                 ))
33         except sqlite3.Error as e:
34             print(f"发生错误: {e}")
35         finally:
36             # 关闭数据库连接
37             if conn:
38                 conn.close()
```

```
1 def print_table_data(table_name, db_path="stocks"):
2     """
3     打印指定 SQLite 数据库中某个表的所有数据，并对齐列。
4
5     参数:
6         table_name (str): 要查看数据的表名。
7         """
8     try:
9         # 连接到 SQLite 数据库
10        conn = sqlite3.connect(db_path)
11        cursor = conn.cursor()
12        # 构建 SQL 查询语句
13        query = f"SELECT * FROM {table_name}"
14        # 执行查询
15        cursor.execute(query)
16        rows = cursor.fetchall()
17        # 获取列名
18        column_names = [description[0] for description in cursor.description]
```

python

```

19         # 获取列类型信息
20         cursor.execute(f"PRAGMA table_info('{table_name}')"")
21         table_info = cursor.fetchall()
22         column_types = [info[2].upper() for info in table_info]
23         # 计算每列的最大宽度
24         col_widths = [len(col) for col in column_names]
25         for row in rows:
26             for idx, item in enumerate(row):
27                 if isinstance(item, float):
28                     item_str = f"{item:.2f}"
29                 else:
30                     item_str = str(item)
31                 col_widths[idx] = max(col_widths[idx], len(item_str))
32
33         # 构建格式化字符串
34         format_str = " | ".join(f"{{: <{width}}} " for width in col_widths)
35         # 打印列名
36         print(f"表 '{table_name}' 的数据: ")
37         print(format_str.format(*column_names))
38         print("-" * (sum(col_widths) + 3 * (len(col_widths) - 1)))
39         # 打印每一行数据
40         for row in rows:
41             formatted_row = []
42             for idx, item in enumerate(row):
43                 if column_types[idx] in ('REAL', 'FLOAT', 'DOUBLE') and isinstance(item, float):
44                     formatted_item = f"{item:.2f}"
45                 else:
46                     formatted_item = str(item)
47             formatted_row.append(formatted_item)
48             print(format_str.format(*formatted_row))
49
50     except sqlite3.Error as e:
51         print(f"发生错误: {e}")
52     finally:
53         # 关闭数据库连接
54         if conn:
55             conn.close()

```

```
1 print_table_schema("my_stock")
```

python

表 'my_stock' 的结构信息:

cid	name	type	notnull	dflt_value	pk
0	stock_id	TEXT	0	None	1
1	volume	INTEGER	1	None	0
2	avg_price	REAL	1	None	0
3	profit	REAL	1	0	0

```
1 print_table_schema("trans")
```

python

表 'trans' 的结构信息:

cid	name	type	notnull	dflt_value	pk
0	trans_id	TEXT	0	None	1

1	stock_id	TEXT	1	None	0
2	date	DATE	1	None	0
3	price	REAL	1	None	0
4	amount	INTEGER	1	None	0
5	sell_or_buy	TEXT	1	None	0

根据要求，我们需要完成 3 个触发器：

1. 拒绝无效卖出交易
2. 买入后更新持仓数量与加权平均价
3. 卖出后更新持仓数量与加权平均价

```

1  # 拒绝无效卖出交易
2  cursor.execute('''
3  DROP TRIGGER IF EXISTS before_insert_trans
4  ''')
5
6  cursor.execute('''
7  CREATE TRIGGER before_insert_trans
8  BEFORE INSERT ON trans
9  WHEN NEW.sell_or_buy = 'sold'
10 BEGIN
11     -- 如果卖出数量超过当前持仓，Abort 并抛错
12     -- 当前甚至没有持仓的话，数量为0
13     SELECT RAISE(ABORT, '卖出数量超过持仓，交易被拒绝')
14     WHERE NEW.amount > COALESCE(
15         (SELECT volume FROM my_stock WHERE stock_id = NEW.stock_id), 0
16     );
17 END;
18 ''')
19 conn.commit()

```

```

1  # 买入后更新持仓数量与加权平均价
2  cursor.execute('''
3  DROP TRIGGER IF EXISTS after_insert_buy
4  ''')
5
6  cursor.execute('''
7  CREATE TRIGGER after_insert_buy
8  AFTER INSERT ON trans
9  WHEN NEW.sell_or_buy = 'buy'
10 BEGIN
11     -- 如果已有持仓，更新 volume 和 avg_price；否则插入新记录
12     INSERT INTO my_stock(stock_id, volume, avg_price, profit)
13     VALUES (
14         NEW.stock_id,
15         NEW.amount,
16         NEW.price,
17         0
18     )
19     ON CONFLICT(stock_id) DO UPDATE SET
20         volume      = volume + NEW.amount,
21         avg_price   = (volume * avg_price + NEW.price * NEW.amount)
22                     / (volume + NEW.amount);

```

```
23 END;
24 '''
25 conn.commit()

1 # 卖出后更新持仓数量与加权平均价
2 # 注意：检查是否存在这只股票的触发器已经在前面定义了
3 cursor.execute('''
4 DROP TRIGGER IF EXISTS after_insert_sell
5 ''')
6
7 cursor.execute('''
8 CREATE TRIGGER after_insert_sell
9 AFTER INSERT ON trans
10 WHEN NEW.sell_or_buy = 'sold'
11 BEGIN
12     -- 扣减持仓
13     UPDATE my_stock
14         SET volume = volume - NEW.amount
15         WHERE stock_id = NEW.stock_id;
16 END;
17 ''')
18 conn.commit()
```

python

由于SQLite3不支持循环等复杂操作，我们在Python中实现先入先出(FIFO)式的匹配股票操作，并且使用Sqlite3的create_function将Python函数暴露给SQL引擎。同时，我们创建一个新的表sale_buy_alloc来进行配对记录。

考虑到在读数据库的时候数据库被上锁的问题，我们新建一个数据库来保存分配信息。

但是问题在于SQLite不支持在触发器中使用UDF。不过，Gunter Hick给出了一个解决方案：“The trigger program is compiled when the CREATE TRIGGER statement is executed (during initial execution or when the schema is loaded from the file). Any functions referenced in triggers need to be defined at that point in time. Your schema will fail to load unless the functions referenced in trigger programs are defined.

You should be able to build an extension that creates your user defined functions and then executes one or more CREATE TEMP TRIGGER statements that use the now-defined functions. See <https://sqlite.org/loadext.html>”。

我们在这里使用这种方案解决SQLite的这个问题。

```
1 conn_alloc = sqlite3.connect("alloc")
2 cursor_alloc = conn_alloc.cursor()
3
4 cursor_alloc.execute('''
5 CREATE TABLE IF NOT EXISTS sale_buy_alloc (
6     sale_trans_id TEXT NOT NULL,
7     buy_trans_id TEXT NOT NULL,
8     alloc_amt INTEGER NOT NULL,
9     PRIMARY KEY (sale_trans_id, buy_trans_id)
10 );
11 ''')
12
13 cursor_alloc.execute('''
14 DELETE FROM sale_buy_alloc
15 ''')
16 conn_alloc.commit()
```

python

```
17 print_table_schema("sale_buy_alloc", db_path="alloc")
```

表 'sale_buy_alloc' 的结构信息:

cid	name	type	notnull	dflt_value	pk
0	sale_trans_id	TEXT	1	None	1
1	buy_trans_id	TEXT	1	None	2
2	alloc_amt	INTEGER	1	None	0

```
1 def calculate_profit(sale_trans_id, stock_id, sell_price, sell_amt):
```

python

```
2     conn = sqlite3.connect('stocks')
```

```
3     conn.row_factory = sqlite3.Row # 让 fetch 回来的是 dict-like 行
```

```
4     cursor = conn.cursor()
```

```
5
```

```
6     profit = 0.0
```

```
7
```

```
8     # 把另一个数据库里的东西挂载进来
```

```
9     cursor.execute("ATTACH DATABASE ? AS allocdb", ('alloc',))
```

```
10
```

```
11     # 查询所有未完全分配的买入记录
```

```
12     cursor.execute("""
```

```
13         SELECT
```

```
14             t.trans_id,
```

```
15             t.price AS buy_price,
```

```
16             t.amount - COALESCE(SUM(a.alloc_amt), 0) AS remain
```

```
17         FROM trans t
```

```
18         LEFT JOIN allocdb.sale_buy_alloc a
```

```
19             ON t.trans_id = a.buy_trans_id
```

```
20         WHERE t.stock_id = ?
```

```
21         AND t.sell_or_buy = 'buy'
```

```
22         GROUP BY t.trans_id, t.price, t.amount
```

```
23         HAVING remain > 0
```

```
24         ORDER BY t.date ASC
```

```
25     """, (stock_id,))
```

```
26
```

```
27     conn_alloc = sqlite3.connect('alloc')
```

```
28     cursor_alloc = conn_alloc.cursor()
```

```
29
```

```
30     for buy in cursor:
```

```
31         if sell_amt <= 0:
```

```
32             break
```

```
33         buy_trans_id, buy_price, remain = buy[0], buy[1], buy[2]
```

```
34         match_amt = min(remain, sell_amt)
```

```
35         profit += (sell_price - buy_price) * match_amt
```

```
36         # 写入分配关系
```

```
37         cursor_alloc.execute("INSERT INTO sale_buy_alloc "
```

```
38             "(sale_trans_id, buy_trans_id, alloc_amt) "
```

```
39             "VALUES (?, ?, ?)",
```

```
40             (sale_trans_id, buy_trans_id, match_amt))
```

```
41         sell_amt -= match_amt
```

```
42
```

```
43     conn_alloc.commit()
```

```
44     conn.commit()
```

```

45
46     conn_alloc.close()
47     conn.close()
48     return profit

```

```

1  # 卖出后计算利润
2  cursor.execute('''
3  DROP TRIGGER IF EXISTS profit_after_buy
4  ''')
5
6  # 注册 UDF, -1表示可以有不限的参数个数
7  conn.create_function("calculate_profit", -1, calculate_profit)
8
9  cursor.execute('''
10 CREATE TEMP TRIGGER profit_after_buy
11 AFTER INSERT ON trans
12 WHEN NEW.sell_or_buy = 'sold'
13 BEGIN
14     -- 调用 Python 注册的 UDF, 执行 FIFO 配对 & 更新 profit 并返回本次 profit
15     UPDATE my_stock
16         SET profit = profit + calculate_profit(NEW.trans_id, NEW.stock_id, NEW.price,
17         NEW.amount)
17     WHERE stock_id = NEW.stock_id;
18 END;
19 ''')
20 conn.commit()

```

现在我们使用提供的触发器的测试数据进行测试。

```

1  # 清空表
2  cursor.execute("DELETE FROM my_stock")
3  cursor.execute("DELETE FROM trans")
4  conn.commit()

```

```

1  def test(trans_id, stock_id, date, price, amount, sell_or_buy):
2      try:
3          cursor.execute("INSERT INTO trans"
4                          "(trans_id, stock_id, date, price, amount, sell_or_buy)"
5                          "VALUES (?, ?, ?, ?, ?, ?)",
6                          (trans_id, stock_id, date, price, amount, sell_or_buy))
7          conn.commit()
8          print_table_data("my_stock")
9          print_table_data("trans")
10     except Exception as e:
11         print("触发器拦截非法操作:", e)
12         conn.rollback()

```

```

1  test('1', '1', '2025-01-01', 10, 1000, 'buy')

```

表 'my_stock' 的数据:

stock_id	volume	avg_price	profit
1	1000	10.00	0.00

表 'trans' 的数据:

trans_id	stock_id	date	price	amount	sell_or_buy
1	1	2025-01-01	10	1000	buy


```
-----  
1          | 1          | 2025-01-01 | 10.00 | 1000    | buy
```

```
1 test('2', '1', '2025-01-02', 11, 500, 'buy')
```

[python](#)

表 'my_stock' 的数据:

stock_id	volume	avg_price	profit
----------	--------	-----------	--------

```
-----  
1          | 1500      | 10.33      | 0.00
```

表 'trans' 的数据:

trans_id	stock_id	date	price	amount	sell_or_buy
----------	----------	------	-------	--------	-------------

```
-----  
1          | 1          | 2025-01-01 | 10.00 | 1000    | buy  
2          | 1          | 2025-01-02 | 11.00 | 500     | buy
```

```
1 test('3', '1', '2025-01-03', 12, 800, 'sold')
```

[python](#)

表 'my_stock' 的数据:

stock_id	volume	avg_price	profit
----------	--------	-----------	--------

```
-----  
1          | 700       | 10.33      | 1600.00
```

表 'trans' 的数据:

trans_id	stock_id	date	price	amount	sell_or_buy
----------	----------	------	-------	--------	-------------

```
-----  
1          | 1          | 2025-01-01 | 10.00 | 1000    | buy  
2          | 1          | 2025-01-02 | 11.00 | 500     | buy  
3          | 1          | 2025-01-03 | 12.00 | 800     | sold
```

```
1 test('4', '1', '2025-01-04', 12.0, 1000, 'sold')
```

[python](#)

触发器拦截非法操作: 卖出数量超过持仓, 交易被拒绝

```
1 test('5', '1', '2025-01-05', 9.0, 1000, 'buy')
```

[python](#)

表 'my_stock' 的数据:

stock_id	volume	avg_price	profit
----------	--------	-----------	--------

```
-----  
1          | 1700      | 9.55       | 1600.00
```

表 'trans' 的数据:

trans_id	stock_id	date	price	amount	sell_or_buy
----------	----------	------	-------	--------	-------------

```
-----  
1          | 1          | 2025-01-01 | 10.00 | 1000    | buy  
2          | 1          | 2025-01-02 | 11.00 | 500     | buy  
3          | 1          | 2025-01-03 | 12.00 | 800     | sold  
5          | 1          | 2025-01-05 | 9.00  | 1000    | buy
```

```
1 test('6', '1', '2025-01-06', 12.0, 800, 'sold')
```

[python](#)

表 'my_stock' 的数据:

stock_id	volume	avg_price	profit
----------	--------	-----------	--------

```
-----  
1          | 900       | 9.55       | 2800.00
```

表 'trans' 的数据:

trans_id	stock_id	date	price	amount	sell_or_buy
----------	----------	------	-------	--------	-------------

```
-----  
1          | 1          | 2025-01-01 | 10.00 | 1000    | buy  
2          | 1          | 2025-01-02 | 11.00 | 500     | buy  
3          | 1          | 2025-01-03 | 12.00 | 800     | sold  
5          | 1          | 2025-01-05 | 9.00  | 1000    | buy  
6          | 1          | 2025-01-06 | 12.00 | 800     | sold
```

```
1 test('7', '1', '2025-01-07', 7.0, 800, 'sold')
```

[python](#)

表 'my_stock' 的数据:

stock_id	volume	avg_price	profit
----------	--------	-----------	--------

1 | 100 | 9.55 | 1200.00

表 'trans' 的数据:

trans_id	stock_id	date	price	amount	sell_or_buy
1	1	2025-01-01	10.00	1000	buy
2	1	2025-01-02	11.00	500	buy
3	1	2025-01-03	12.00	800	sold
5	1	2025-01-05	9.00	1000	buy
6	1	2025-01-06	12.00	800	sold
7	1	2025-01-07	7.00	800	sold

1 print_table_data("sale_buy_alloc", db_path="alloc")

python

表 'sale_buy_alloc' 的数据:

sale_trans_id	buy_trans_id	alloc_amt
3	1	800
6	1	200
6	2	500
6	5	100
7	5	800